

ECOM EXPERTS

Ps translationfinder

Manual de usuario, tecnico y de desarrollo

ps_translationfinder 1.0.5 · 07/09/2026

Contenido

¿Qué hace?

Instalación

Uso del buscador

Cambiar una traducción

Sintaxis de búsqueda (comodín %)

Buscar y reemplazar (masivo)

Contenido de la tienda (tablas *_lang) — opcional

Backup e historial de reemplazos (restaurar)

Botones útiles

Preguntas frecuentes

Modo depuración

1. Propósito

2. Cómo guarda PrestaShop las traducciones (base del diseño)

3. Arquitectura

4. Flujo

5. Rendimiento

6. Seguridad / estándares

7. Configuración

1. Puntos de extensión

Añadir/ajustar la clasificación de apartados

Añadir nuevas raíces de búsqueda

Cambiar el límite de resultados

2. Contrato de una "fila" (row)

3. Persistencia (TranslationSaver)

4. Reglas PrestaShop respetadas

5. Depuración

6. Cómo añadir un endpoint AJAX

7. Pendientes / mejoras futuras

¿Qué hace?

Te permite **buscar cualquier texto** de tu tienda y saber **dónde aparece** (front, back office, módulos, theme o emails), y **cambiarlo** desde un único buscador, sin tener que recorrer todos los paneles de traducción de PrestaShop.

Instalación

1. Copia la carpeta `ps_translationfinder` en `/modules`.
2. En el back office: **Módulos** → **Gestor de módulos**, busca "Translation Finder" e **instala**.
3. Se crea un acceso en el menú **Localización** → **Buscar traducción** (y un botón desde la configuración del módulo).

Uso del buscador

1. Abre **Localización** → **Buscar traducción**.
2. Escribe el texto que buscas (mínimo **3 caracteres**) y elige el **idioma**.
3. Pulsa **Buscar**.
4. A la **izquierda** filtra por apartado: - **Todos** - **Front / Theme** (lo que ve el cliente) - **Back office** (administración) - **Módulos** - **Emails** - **Legacy / Otros**
5. En la tabla de resultados verás, por cada coincidencia: el **apartado**, el **dominio y archivo** donde está, el **texto original** y la **traducción actual** (editable).

Cambiar una traducción

1. Edita el texto en la columna **Traducción actual**.
2. Pulsa **Guardar** en esa fila.
3. El cambio se guarda y se limpia la caché automáticamente. Recarga la página de la tienda para verlo.

Front, back, módulos y theme: el cambio se guarda en la base de datos (método oficial de PrestaShop), **sin tocar archivos del core**. Es seguro y sobrevive a actualizaciones del catálogo.

Emails: al ser plantillas, el texto se cambia directamente en el archivo del email. El módulo crea automáticamente una **copia de seguridad** antes de modificarlo.

Sintaxis de búsqueda (comodín %)

- `palabra` → busca **solo esa palabra/frase exacta**.
- `%palabra` → cualquier cosa + palabra + nada (**termina en**).

- `palabra%` → palabra + cualquier cosa (**empieza por**).
- `%palabra%` → cualquier cosa + palabra + cualquier cosa (**contiene**).

Buscar y reemplazar (masivo)

1. Busca el texto (puedes usar comodines).
2. En el campo "**Reemplazar el texto buscado por...**" escribe el texto nuevo.
3. Pulsa **Reemplazar en todos los resultados** y confirma.
4. Sustituye la palabra/frase en la traducción de **todas** las coincidencias del apartado seleccionado y guarda los cambios (la caché se limpia al terminar). El reemplazo usa la palabra literal, ignorando los `%`.

Contenido de la tienda (tablas `*_lang`) — opcional

Por defecto el buscador trabaja con traducciones de **interfaz**. Si quieres buscar/editar también **contenido** (nombres de productos, categorías, páginas CMS, bloques de texto...): 1. Ve a la **configuración** del módulo y activa "**Buscar también en CONTENIDO de BD**". 2. Aparecerá el apartado "**Contenido (BD)**" en la columna izquierda. 3. Selecciónalo y busca: encontrará el texto en cualquier tabla `*_lang` y podrás editarlo.

Edita contenido real de la tienda directamente en BD; úsalo con cuidado. No se incluye en "Todos" por rendimiento: hay que elegir el apartado expresamente.

Backup e historial de reemplazos (restaurar)

- Junto al botón de reemplazo hay una casilla "**Backup (restaurable)**" (activada por defecto, configurable).
- Con el backup activo, cada **reemplazo masivo** guarda los valores anteriores.
- Debajo de los resultados verás el **Historial de reemplazos** con: fecha, apartado, texto buscado, reemplazo, nº de cambios y un botón **Restaurar** para deshacer ese reemplazo (devuelve los textos a su valor anterior). También puedes eliminar una entrada del historial (no revierte, solo la quita de la lista).

Botones útiles

- **Limpiar caché**: fuerza el refresco si un cambio no se ve reflejado.
- El buscador limita a **300 resultados**; si te avisa de que hay más, afina el texto.

Preguntas frecuentes

- **No encuentro un texto**. Prueba con menos palabras, revisa el idioma seleccionado y el apartado "Todos".
- **Cambié un email y no se aplica**. Comprueba que la carpeta `mails/` tiene permisos de escritura.

- **El cambio no aparece en la tienda.** Pulsa **Limpiar caché** y recarga.

Modo depuración

En la configuración del módulo puedes activar **Modo depuración**: genera registros en la carpeta `/logs` del módulo, útiles para soporte técnico.

Translation Finder — Documentación Técnica

Versión: 1.0.0 · **Autor:** Ecom Experts · **Licencia:** AFL-3.0 · **Compatibilidad:** PrestaShop 1.7 / 8 / 9 (PHP 7.4 y 8.x)

1. Propósito

Buscador unificado de traducciones. Localiza cualquier texto en **front/theme, back office, módulos y emails**, indica **dónde aparece** (dominio + archivo) y permite **editarlo** sin overrides ni modificar el core.

2. Cómo guarda PrestaShop las traducciones (base del diseño)

- **Sistema moderno (Symfony):** catálogo base en archivos **XLIFF** `.xlf` (`<source>` / `<target>`). Las traducciones editadas en el BO se guardan en la tabla `ps_translation` y **tienen prioridad** sobre los `.xlf`.
- **Dominio en BD:** se almacena **sin puntos** (`Admin.Advparameters.Feature` → `AdminAdvparametersFeature`), idéntico al atributo `original` / nombre del fichero XLIFF. El módulo usa exactamente ese valor para que el guardado surta efecto.
- **key** = texto fuente (source) · **translation** = traducción · **theme** = nombre del theme (front) o `NULL` (core/admin/módulos).
- **Legacy:** `modules/<m>/translations/<iso>.php` con array `$_MODULE` (hashes). **Emails:** plantillas `.html` / `.txt` en `mails/<iso>/`.

3. Arquitectura

<code>ps_translationfinder.php</code>	Clase <code>Ps_Translationfinder</code> (Module): <code>install/tab,</code>
<code>getContent, addnewfeatures</code>	
<code>classes/</code>	
<code>TranslationCatalog.php</code>	Constantes de categoría/origen, clasificación de
dominio, normalización de filas	
<code>TranslationScanner.php</code>	Rastrea XLIFF + legacy + emails + <code>ps_translation</code> ; filtra
por término/categoría	
<code>TranslationSaver.php</code>	Persiste: BD (<code>ps_translation</code>) email (archivo+backup)
legacy (.php)	
<code>TranslationfinderLogger.php</code>	Log opcional a <code>/logs</code> si <code>PS_TFINDER_DEBUG=1</code>
<code>controllers/admin/</code>	
<code>AdminPsTranslationfinderController.php</code>	Pinta el buscador y atiende AJAX
(Search/Save/ClearCache)	
<code>views/templates/admin/</code>	<code>search.tpl</code> (UI) · <code>configure.tpl</code> (config)
<code>views/js/ps_translationfinder.js</code> · <code>views/css/ps_translationfinder.css</code>	
<code>translations/es-ES/ModulesPstranslationfinderAdmin.es-ES.xlf</code>	
<code>logs/</code> · <code>cache/backups/</code> · <code>docs/</code>	

No crea tablas propias: reutiliza `ps_translation`. `index.php` (header + redirect) en cada carpeta.

4. Flujo

1. **Buscar** (`ajaxProcessSearch`): `TranslationScanner::search($term, $category, 300)` carga el mapa de `ps_translation`, recorre los XLIFF de la categoría (core `translations/<locale>`, `app/Resources/translations/<locale>` y `default`, `theme` y módulos), añade `legacy` y emails, fusiona el valor vigente de BD y devuelve filas normalizadas (con `id`, `origin`, `category`, `domain`, `source`, `translation`, `file`, `theme`).
2. **Guardar** (`ajaxProcessSave`): `TranslationSaver::save($row, $value)` decide por `origin`: - `file` / `db` → `INSERT/UPDATE` en `ps_translation` (busca por `id_lang` + `domain` + `key` + `theme`) y limpia caché. - `email` → backup en `cache/backups/` y `str_ireplace` del texto en el archivo. - `legacy` → reescribe el array `$_MODULE` del `.php`.
3. **Limpiar caché** (`ajaxProcessClearCache`): `Tools::clearAllCache()` + `Tools::clearSf2Cache()` (si existe).

5. Rendimiento

- Mínimo 3 caracteres; búsqueda bajo demanda (botón), no en cada tecla.
- Resultados limitados a **300** (`truncated=true` avisa de refinar).
- Corte temprano al alcanzar el límite. Rutas siempre con `_PS_ROOT_DIR_`, `_PS_MODULE_DIR_`, `_PS_MAIL_DIR_`, `_PS_ALL_THEMES_DIR_`.

6. Seguridad / estándares

- `if (!defined('_PS_VERSION_')) exit;` en todos los PHP; headers AFL-3.0; sin BOM; sin Composer; carga manual `require_once`.
- `pSQL()` en todo SQL manual; sin `LIMIT 1`; escape Smarty `|escape: 'html': 'UTF-8'`; tokens de admin en AJAX (sin estado de sesión).
- `isUsingNewTranslationSystem()` devuelve `true`.

7. Configuración

- `PS_TFINDER_DEBUG` (0/1): activa logs en `/logs/ps_translationfinder-YYYY-MM-DD.log`.

Translation Finder — Guía de Desarrollo

Versión: 1.0.0 · **Autor:** Ecom Experts · **Licencia:** AFL-3.0

1. Puntos de extensión

Añadir/ajustar la clasificación de apartados

`classes/TranslationCatalog.php` → `classifyDomain($domain)`. El orden importa: `Emails / Mail` → `Admin` → `Modules` → `Shop` → otros. Las categorías de la UI se definen en `getCategories()`.

Añadir nuevas raíces de búsqueda

`classes/TranslationScanner.php` → `getXliffJobs($category)` devuelve `['glob' => patrón, 'theme' => nombre]`. Añade ahí nuevas rutas (p.ej. otra carpeta de catálogos). Para emails edita `scanEmails()` (lista `$dirs`).

Cambiar el límite de resultados

Controlador `ajaxProcessSearch()` → tercer argumento de `search()` (por defecto 300).

2. Contrato de una "fila" (row)

`TranslationCatalog::makeRow()` normaliza a:

<code>id</code>	<code>string</code>	hash único (base64 de origin domain source file theme)
<code>origin</code>	<code>string</code>	file db email legacy
<code>category</code>	<code>string</code>	all front back modules emails other
<code>domain</code>	<code>string</code>	dominio SIN puntos (o "mail:..." / "legacy:...")
<code>source</code>	<code>string</code>	texto fuente (o hash legacy, o término en emails)
<code>translation</code>	<code>string</code>	traducción vigente (BD si existe, si no el target del XLIFF)
<code>file</code>	<code>string</code>	ruta relativa a la raíz de PrestaShop
<code>theme</code>	<code>string</code>	nombre del theme o '' (global)
<code>editable</code>	<code>bool</code>	

El front guarda el array `lastRows` y referencia por índice (`data-idx`) — no se incrusta JSON en el DOM.

3. Persistencia (TranslationSaver)

- **BD:** se localiza la fila con `id_lang + domain + key + theme` (theme `IS NULL` o `= valor`); si existe `UPDATE`, si no `INSERT`. `key / translation` con `pSQL($v, true)` (permite HTML). Tras escribir → `clearCache()`.
- **Email:** `backupFile()` copia el original a `cache/backups/` con timestamp; luego `str_ireplace(source → value)`. Requiere permisos de escritura en `mails/`.

- **Legacy:** captura `global $_MODULE` , modifica la clave y reescribe el `.php` completo.

4. Reglas PrestaShop respetadas

- Sin overrides (clases propias en `/classes` , controlador en `/controllers/admin`).
- Sin namespace en la clase principal; nombre `Ps_Translationfinder` = carpeta.
- Sin dependencias Composer; funciones nativas (`SimpleXML` , `glob`).
- Tab instalada en `install()` vía objeto `Tab` (no `installOverrides`).
- `addnewfeatures()` con `try{}` reinstala la tab si falta, al entrar en configuración.
- Compatibilidad PHP 7.4/8: sin `::class` , `\Context::getContext()` , comprobaciones defensivas.

5. Depuración

Activa **Modo depuración** en la configuración (`PS_TFINDER_DEBUG`). `TranslationfinderLogger::log($msg, $context)` escribe en `/logs` . Errores de scanner/saver/backup se registran sin romper el flujo.

6. Cómo añadir un endpoint AJAX

En `AdminPsTranslationfinderController` crea `ajaxProcessMiAccion()` y llama desde JS con `data: { ajax:1, action:'MiAccion', ... }` . Responde con `$this->ajaxDie(json_encode([...]))` .

7. Pendientes / mejoras futuras

- Índice cacheado en `/cache` para acelerar búsquedas repetidas (estructura ya prevista).
- Soporte legacy de theme (`themes/<t>/lang/<iso>.php`).
- Exportar/importar cambios; historial de ediciones.